

```

1  import components.simplereader.SimpleReader;
2
3  /**
4   * Program to convert an XML RSS (version 2.0) feed from a given URL into the
5   * corresponding HTML output file.
6   *
7   * @author Eric Zhou
8   */
9  public final class RSSAggregator {
10
11      /**
12       * Private constructor so this utility class cannot be instantiated.
13       */
14      private RSSAggregator() {
15      }
16
17      /**
18       * Outputs the "opening" tags in the generated HTML file. These are the
19       * expected elements generated by this method:
20       *
21       * <html> <head> <title>the channel tag title as the page title</title>
22       * </head> <body>
23       * <h1>the page title inside a link to the <channel> link</h1>
24       * <p>
25       * the channel description
26       * </p>
27       * <table border="1">
28       * <tr>
29       * <th>Date</th>
30       * <th>Source</th>
31       * <th>News</th>
32       * </tr>
33       *
34       * @param channel
35       *         the channel element XMLTree
36       * @param out
37       *         the output stream
38       * @updates out.content
39       * @requires [the root of channel is a <channel> tag] and out.is_open
40       * @ensures out.content = #out.content * [the HTML "opening" tags]
41       */
42      private static void outputHeader(XMLTree channel, SimpleWriter out) {
43          assert channel != null : "Violation of: channel is not null";
44          assert out != null : "Violation of: out is not null";
45          assert channel.isTag() && channel.label().equals("channel")
46              : "" + "Violation of: the label root of channel is a <channel> tag";
47          assert out.isOpen() : "Violation of: out.is_open";
48
49          int titleIndex = getChildElement(channel, "title"); //find the child
50          int linkIndex = getChildElement(channel, "link");
51          int descriptionIndex = getChildElement(channel, "description");
52
53          String title; //check the title child
54          if (titleIndex != -1 && channel.child(titleIndex).numberOfChildren() > 0) {
55              title = channel.child(titleIndex).child(0).label();
56          } else {
57              title = "Empty Title";
58          }
59
60          String link = channel.child(linkIndex).child(0).label();
61
62          String description; //check the description
63          if (descriptionIndex != -1
64              && channel.child(descriptionIndex).numberOfChildren() > 0) {

```

```

71      description = channel.child(descriptionIndex).child(0).label();
72  } else {
73      description = "No Description";
74  }
75
76  out.println("<html>"); //print the header
77  out.println("<head><title>" + title + "</title></head>");
78  out.println("<body>");
79  out.println("<h1><a href=\"\" + link + "\">" + title + "</a></h1>");
80  out.println("<p>" + description + "</p>");
81  out.println("<table border =\"1\">");
82  out.println("<tr><th>Date</th><th>Source</th><th>News</th></tr>");
83  }
84
85  /**
86   * Outputs the "closing" tags in the generated HTML file. These are the
87   * expected elements generated by this method:
88   *
89   * </table>
90   * </body> </html>
91   *
92   * @param out
93   *         the output stream
94   * @updates out.contents
95   * @requires out.is_open
96   * @ensures out.content = #out.content * [the HTML "closing" tags]
97   */
98  private static void outputFooter(SimpleWriter out) {
99      assert out != null : "Violation of: out is not null";
100     assert out.isOpen() : "Violation of: out.is_open";
101
102     out.println("</table>");
103     out.println("</body></html>");
104 }
105
106 /**
107  * Finds the first occurrence of the given tag among the children of the
108  * given {@code XMLTree} and return its index; returns -1 if not found.
109  *
110  * @param xml
111  *        the {@code XMLTree} to search
112  * @param tag
113  *        the tag to look for
114  * @return the index of the first child of type tag of the {@code XMLTree}
115  *         or -1 if not found
116  * @requires [the label of the root of xml is a tag]
117  * @ensures <pre>
118  * getChildElement =
119  * [the index of the first child of type tag of the {@code XMLTree} or
120  * -1 if not found]
121  * </pre>
122  */
123  private static int getChildElement(XMLTree xml, String tag) {
124      assert xml != null : "Violation of: xml is not null";
125      assert tag != null : "Violation of: tag is not null";
126      assert xml.isTag() : "Violation of: the label root of xml is a tag";
127
128      int index = -1;
129      for (int i = 0; i < xml.numberOfChildren() && index == -1; i++) {
130          if (xml.child(i).label().equals(tag)) { //check every tag
131              index = i;
132          }
133      }
134      return index;
135  }

```

```

136
137 /**
138  * Processes one news item and outputs one table row. The row contains three
139  * elements: the publication date, the source, and the title (or
140  * description) of the item.
141  *
142  * @param item
143  *         the news item
144  * @param out
145  *         the output stream
146  * @updates out.content
147  * @requires [the label of the root of item is an <item> tag] and
148  *         out.is_open
149  * @ensures <pre>
150  * out.content = #out.content *
151  * [an HTML table row with publication date, source, and title of news item]
152  * </pre>
153  */
154 private static void processItem(XMLTree item, SimpleWriter out) {
155     assert item != null : "Violation of: item is not null";
156     assert out != null : "Violation of: out is not null";
157     assert item.isTag() && item.label().equals("item")
158         : "" + "Violation of: the label root of item is an <item> tag";
159     assert out.isOpen() : "Violation of: out.is_open";
160
161     int dateIndex = getChildElement(item, "pubDate"); //get the index
162     int titleIndex = getChildElement(item, "title");
163     int linkIndex = getChildElement(item, "link");
164     int sourceIndex = getChildElement(item, "source");
165     int descriptionIndex = getChildElement(item, "description");
166
167     String date;
168     if (dateIndex != -1 && item.child(dateIndex).numberOfChildren() > 0) {
169         date = item.child(dateIndex).child(0).label();
170     } else {
171         date = "No date available";
172     }
173
174     String title;
175     if (titleIndex != -1 && item.child(titleIndex).numberOfChildren() > 0) {
176         title = item.child(titleIndex).child(0).label();
177     } else if (descriptionIndex != -1
178         && item.child(descriptionIndex).numberOfChildren() > 0) {
179         title = item.child(descriptionIndex).child(0).label();
180     } else {
181         title = "No title available";
182     }
183
184     String link = "";
185     if (linkIndex != -1 && item.child(linkIndex).numberOfChildren() > 0) {
186         link = item.child(linkIndex).child(0).label();
187     }
188
189     out.println("<tr>");
190     out.println("<td>" + date + "</td>");
191     if (sourceIndex != -1 && item.child(sourceIndex).numberOfChildren() > 0) {
192         String sourceUrl = item.child(sourceIndex).attributeValue("url");
193         String sourceLabel = item.child(sourceIndex).child(0).label();
194         out.println("<td><a href=\"\" + sourceUrl + \"\">\" + sourceLabel + "</a></td>");
195     } else {
196         out.println("<td>No source available</td>");
197     }
198     if (!link.isEmpty()) {
199         out.println("<td><a href=\"\" + link + \"\">\" + title + "</a></td>");
200     } else {

```

```

201         out.println("<td>" + title + "</td>");
202     }
203     out.println("</tr>");
204 }
205
206 /**
207  * Processes one XML RSS (version 2.0) feed from a given URL converting it
208  * into the corresponding HTML output file.
209  *
210  * @param url
211  *     the URL of the RSS feed
212  * @param file
213  *     the name of the HTML output file
214  * @param out
215  *     the output stream to report progress or errors
216  * @updates out.content
217  * @requires out.is_open
218  * @ensures <pre>
219  * [reads RSS feed from url, saves HTML document with table of news items
220  *  to file, appends to out.content any needed messages]
221  * </pre>
222  */
223 private static void processFeed(String url, String file, SimpleWriter out) {
224     XMLTree xml = new XMLTree1(url);
225     if (xml.label().equals("rss") && xml.attributeValue("version").equals("2.0")) {
226         XMLTree channel = xml.child(0);
227         SimpleWriter fileOut = new SimpleWriter1L(file);
228         outputHeader(channel, fileOut);
229         for (int i = 0; i < channel.numberOfChildren(); i++) {
230             if (channel.child(i).label().equals("item")) {
231                 processItem(channel.child(i), fileOut);
232             }
233         }
234         outputFooter(fileOut);
235         fileOut.close();
236         out.println("generate successfully");
237     } else {
238         out.println("Invalid RSS feed");
239     }
240 }
241
242 /**
243  * @param feeds
244  *     the input file
245  * @param out
246  *     the output stream
247  */
248 private static void outputIndexPage(XMLTree feeds, SimpleWriter out) {
249     String title = feeds.attributeValue("title");
250     out.println("<html>");
251     out.println("<head><title>" + title + "</title></head>");
252     out.println("<body>");
253     out.println("<h1>" + title + "</h1>");
254     out.println("<ul>");
255
256     for (int i = 0; i < feeds.numberOfChildren(); i++) {
257         XMLTree feed = feeds.child(i);
258         String name = feed.attributeValue("name");
259         String file = feed.attributeValue("file");
260         out.println("<li><a href=\"\" + file + \"\">\" + name + "</a></li>");
261     }
262     out.println("</ul>");
263     out.println("</body></html>");
264 }
265

```

```

266 /**
267  * Main method.
268  *
269  * @param args
270  *         the command line arguments; unused here
271  */
272  public static void main(String[] args) {
273      SimpleReader in = new SimpleReader1L();
274      SimpleWriter out = new SimpleWriter1L();
275
276      out.println("Enter the name of the input XML file: ");
277      String inputName = in.nextLine();
278      out.println("Enter the name of the index HTML file: ");
279      String indexName = in.nextLine();
280      XMLTree feeds = new XMLTree1(inputName);
281      SimpleWriter indexOut = new SimpleWriter1L(indexName);
282      outputIndexPage(feeds, indexOut);
283      indexOut.close();
284
285      for (int i = 0; i < feeds.numberOfChildren(); i++) {
286          XMLTree feed = feeds.child(i);
287          String url = feed.attributeValue("url");
288          String file = feed.attributeValue("file");
289          processFeed(url, file, out);
290      }
291
292      in.close();
293      out.close();
294  }
295
296 }
297

```